



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

NSCSCC 2025 – 第九届“龙芯杯”

西北工业大学四队决赛提交作品报告

团队成员：严骄阳、李子谦、上官福炜、乔鑫

指导教师：王党徽、王继禾

日期：2025 年 8 月 19 日

目录

1 概述	2
2 总体设计方案	2
2.1 微架构设计	2
2.2 精确例外机制	2
3 设计优化	4
3.1 全同步设计	4
3.2 组合逻辑优化	4
3 Cache 设计	5
3.1 ICache	5
3.2 DCache	5
4 TLB 设计	6
5 系统展示	7
6 参考设计说明	7
参考文献	8

1 概述

本设计是一个基于 LoongArch32 位精简版指令集^[1]，采用顺序双发射八级流水线架构，频率可达 90Mhz 的处理器软核。本设计能够通过比赛初赛提供的功能测试的 58 个测试点，性能测试分数为 2.142，并正常启动 Linux 操作系统^[2]。

本设计实现了除浮点外全部整数、特权以及 `cpucfg` 指令，支持例外与中断机制，实现了除浮点相关外全部例外，设计了 CRMD 等 36 项控制状态寄存器。

设计集成了 AXI 总线、Cache 和内存管理单元 (MMU)，并实现了分支预测。

处理器包含的指令 Cache 与数据 Cache 均为两路组相联设计，每路 256 项，一项 128 位，共 8KiB。TLB 为二级 TLB 设计，第一级 ITLB 和 DTLB 均使用 4 项直接映射结构，第二级混合 TLB 使用 16 项全相联结构，支持 4K 页和 4M 页。

2 总体设计方案

2.1 微架构设计

本设计采用双发射八级流水线架构，八个流水级分别称为预取指 (PIF)、取指 (IF)、译码 (ID)、缓冲队列 (FIFO)、发射 (IS)、执行 1 (EX1)、执行 2 (EX2)、访存 (MEM)。

指令处理流程始于 PIF 级，其基于跳转信号、分支预测结果或顺序地址预先读取取指信息；IF 级依据取指信息从 `icache` 中读取当前两条指令；读取的指令进入 ID 级进行解析，识别操作码、立即数等信号，并生成相应的控制信号与操作数需求信息；译码后的指令信息被送入 FIFO 级缓存；发射 (IS) 级则依据指令类型、操作数就绪状态及执行单元可用性，动态选择并分发最多两条满足条件的指令至执行级；EX1 级接收指令完成核心操作并将信号传递至 EX2 级；EX2 级完成数据访存处理和 `csr` 数据读取操作；最后 MEM 级进行寄存器写回、`csr` 和 `tlb` 管理等操作；至此，指令完成其核心流水处理过程。

2.2 精确例外机制

精确例外机制的实现包含例外检查、例外报出和例外返回三部分。

IF、ID、IS、EX1、EX2 五级各负责检查出特定的一些例外，并将例外标记和例外编码随着指令沿着流水线向后传播。在 MEM 级访问控制状态寄存器时，处理器报出例外，对 MEM 以及之前的流水级均进行 `flush`，并跳转到例外处理子程序入口地址处继续执行。例外返回时，由软件写入返回地址，以决定是否重新执行报例外的指令，执行 `ertn` 指令后跳回返回地址继续正常的执行流程。

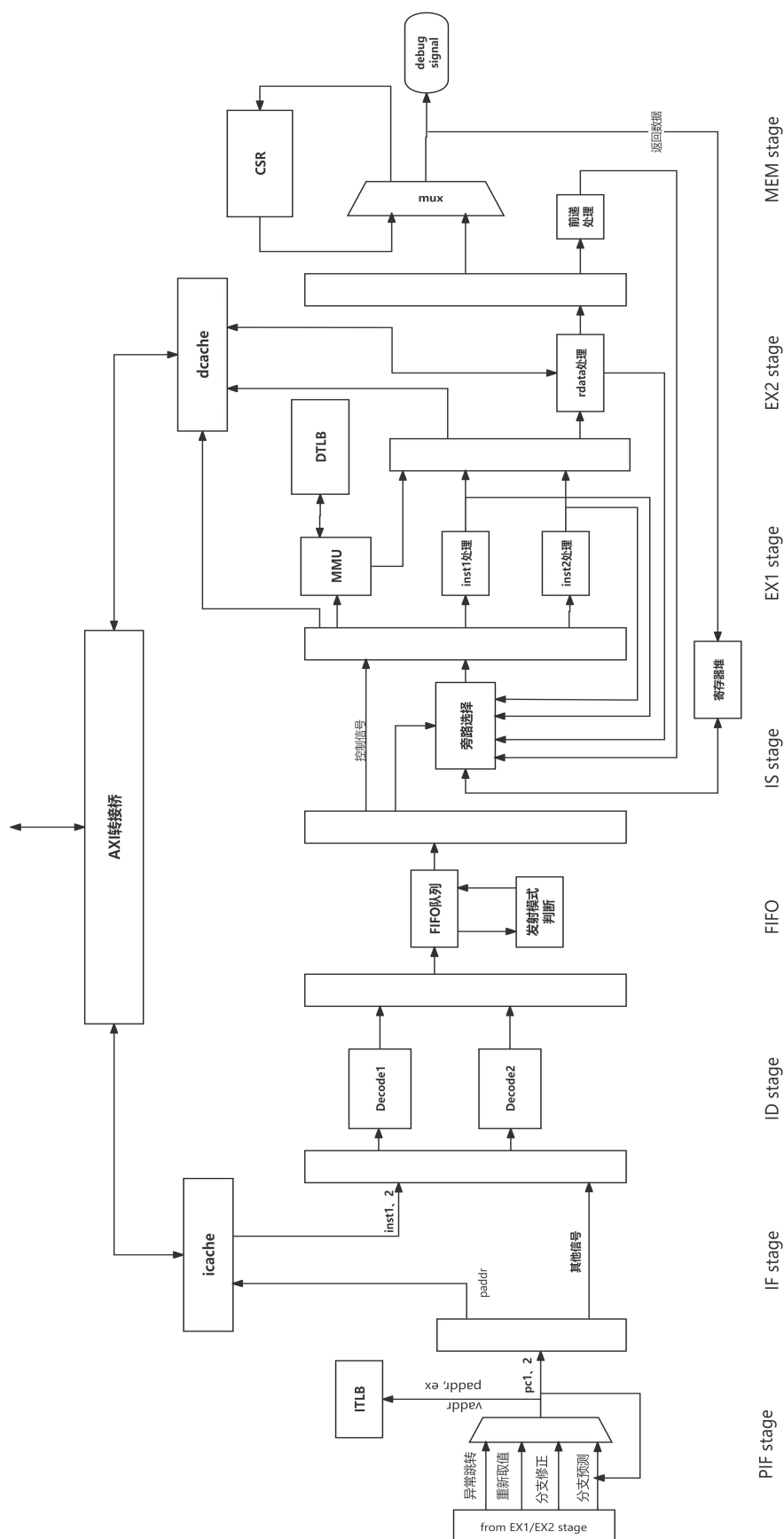


图 1 处理器微架构设计图

3 设计优化

3.1 全同步设计

处理器采用全同步设计，每一级产生的阻塞信号均能阻塞前面所有流水级。主流水线的流水级间不采用握手反压机制，以消除延迟较长的链式 **allowin** 信号，从而提升频率。

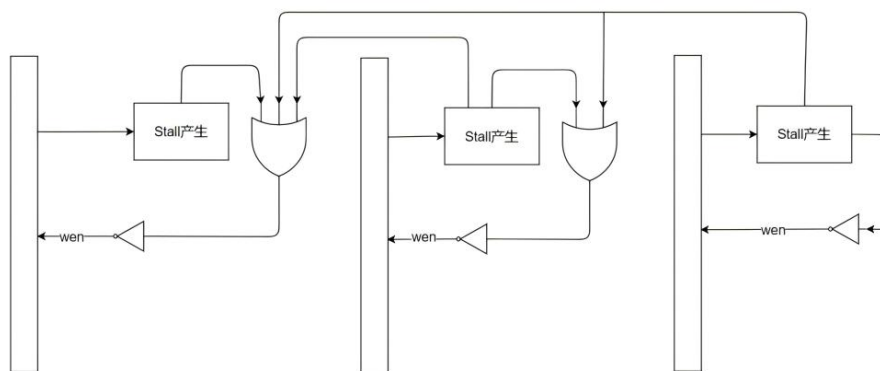


图 2 全同步设计示意图

3.2 逻辑优化

为降低关键路径延迟并提升处理器工作频率，实施了以下组合逻辑优化措施：

- 1、调整 CSR 读取时机：将控制状态寄存器的读取操作由 MEM 提前至 EX2，减少数据通路的组合逻辑深度。
- 2、简化 TLB/CSR 写操作处理：对于转 TLB 和控制状态寄存器写操作，采用直接刷新流水线并 **refetch** 策略，避免使用复杂的前递逻辑。
- 3、拆分取指流水级：将原有的 Fetch 阶段拆分为 PIF 和 IF 两级流水级，分散组合逻辑负载。
- 4、延迟 BTB 更新：将 BTB 的操作推迟至执 EX2 进行，有效缩短相关预测逻辑的关键路径。
- 5、cache 采用 **valid** 和 **cancel** 的组合控制，将发送 **valid** 的流水级中过长组合逻辑的异常信号移到下一流水级，用来发送 **cancel** 信号取消访问，从而避免了 **valid** 信号和异常信号的组合逻辑叠加。

4 Cache 设计

4.1 ICache

本设计的 iCache 采用 2 路组相联设计，配备双读端口，采用 128 位 Cache 行（每路由 4 个 32 位 Bank 组成），支持 CACOP 指令以实现 Cache 维护操作；替换策略上，采用 23 位 LFSR 实现伪随机替换，曾尝试过伪 Lru 但效果一般，故未采用。

指令 Cache 状态设计为四状态转换，如下图所示：

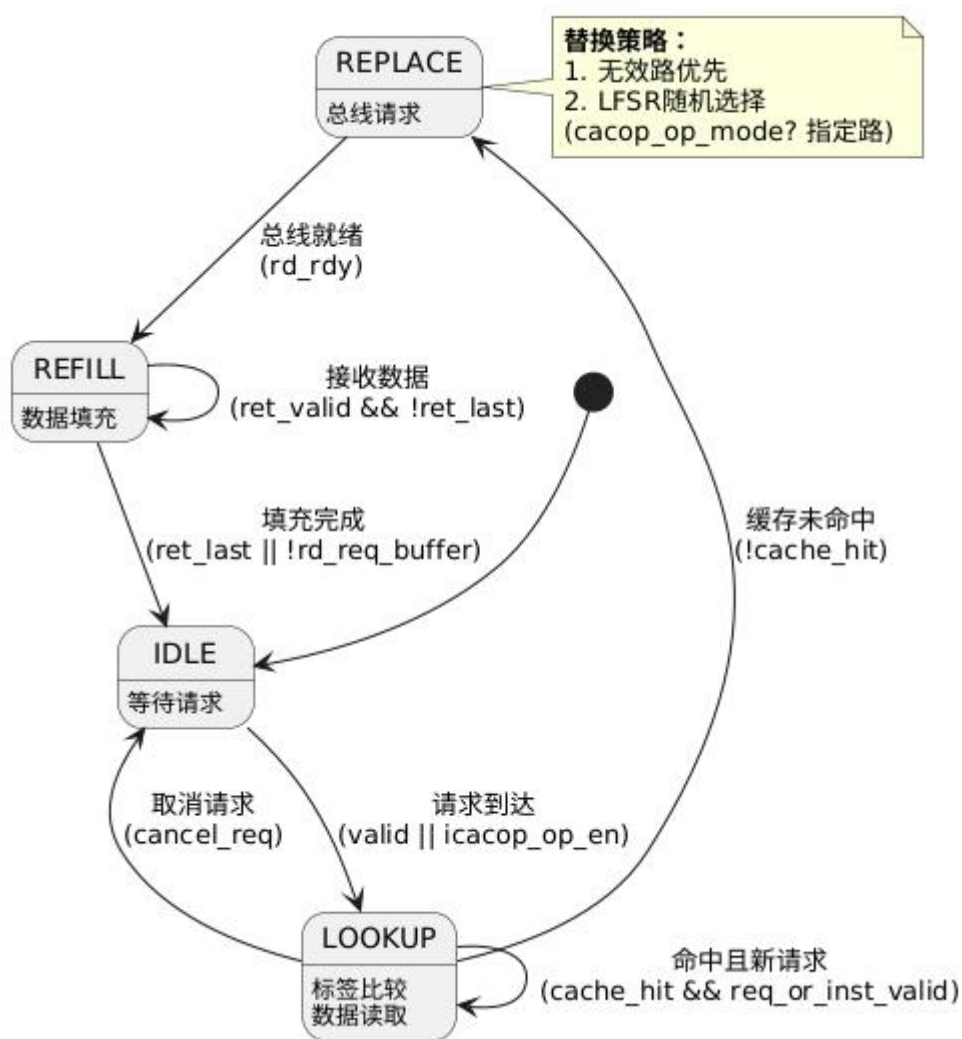


图 3 icache 状态图

4.2 DCache

本设计的 dCache 采用 2 路组相联，配备单读端口，采用 128 位 Cache 行（每路由 4 个 32 位 Bank 组成），支持 CACOP 指令以实现 Cache 维护操作，并实现了 preld 指令；替换策略上，使用 23 位 LFSR 实现伪随机替换。

数据 Cache 状态设计为五状态转换，实现的 writebuffer 为两状态转换，如下图所示：

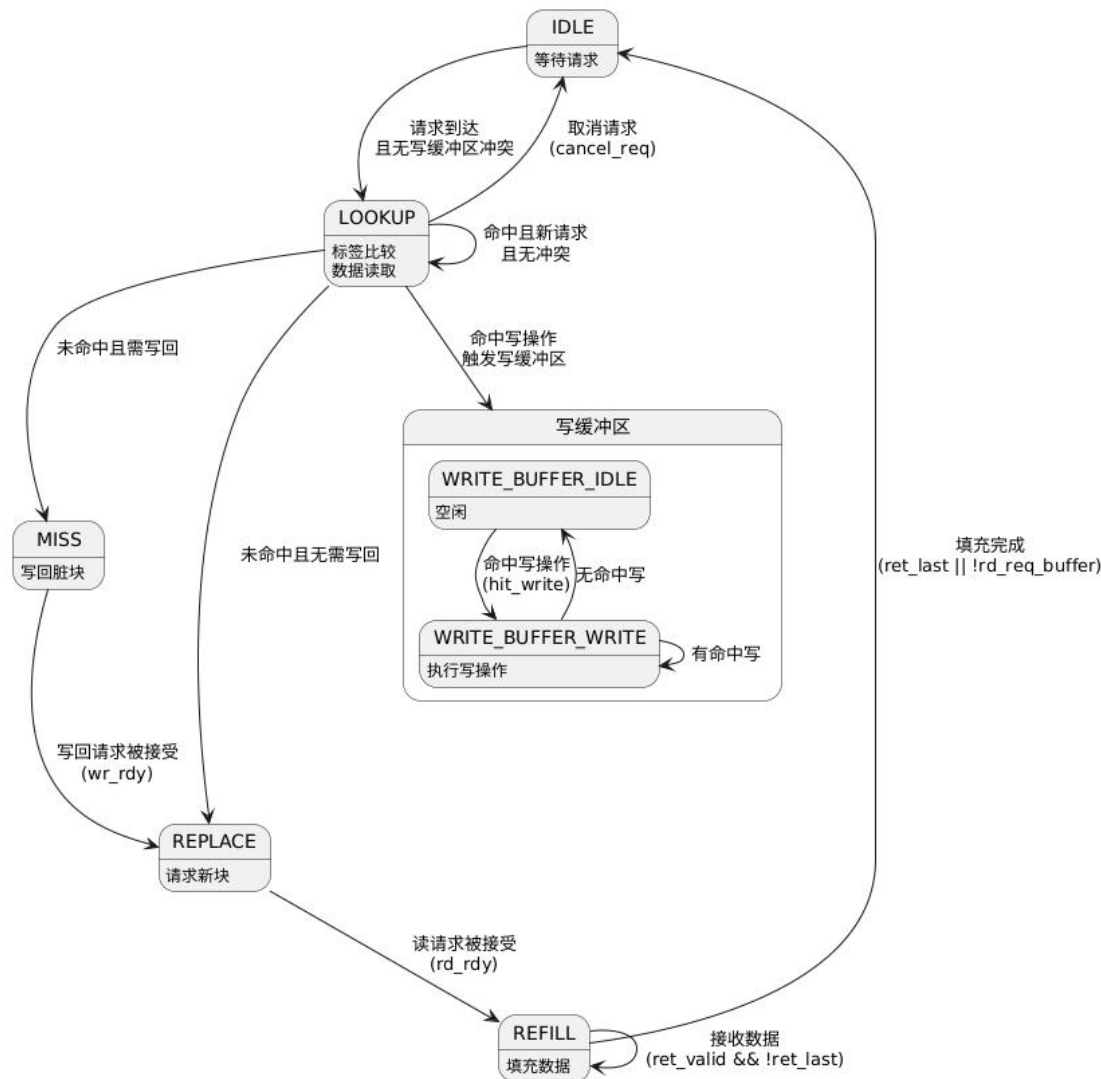


图 4 dcache 状态图

5 TLB 设计

本设计的 TLB 设计为二级 TLB 架构，包含 L1 TLB 和 L2 TLB。L1TLB 设计为 iTLB 与 dTLB 单独实例化，L2TLB 设计为统一缓存，实现了高效的并行查找和状态机控制机制。

本设计第一级 iTLB 和 DTLB 均使用 4 项直接映射结构，第二级混合 TLB 使用 16 项全相联结构，支持 4K 页和 4M 页。第二级混用 TLB 项数在微架构中采用宏定义的方式，曾尝试过 32 项、16 项、8 项、4 项、2 项等不同项数，最终进行多次性能测试分数对比后采用 16 项 L2TLB 设计。

iTLB 与 dTLB 状态设计均为四状态转换，如下图所示：

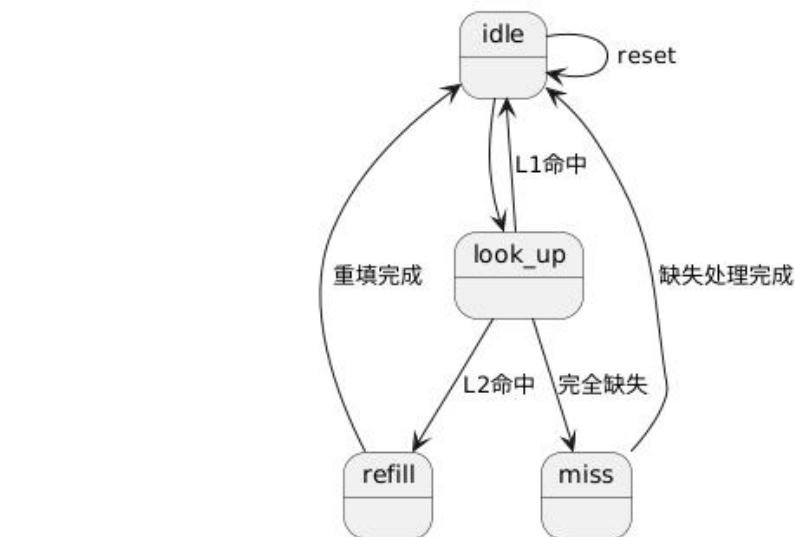


图 5 TLB 状态转换图

6 系统展示

本处理器可通过 u-boot 网络加载并运行内核，使用的内核镜像为龙芯开源项目 la32r-Linux 中发行版 v0.1 版本的 vmlinux 镜像^[9]，具体操作可见 Chiplab 中的介绍^[2]。

```

0.064000 random: get_random_u32 called from bucket_table_alloc.isra.34+0x68/0x1f0 with crng_init=0
0.068000 clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 7645041785100000 ns
0.072000 futex hash table entries: 256 (order: -1, 3072 bytes, linear)
0.076000 NET: Registered PF_NETLINK/PF_ROUTE protocol family
0.244000 pps_core: LinuxPPS API ver. 1 registered
0.244000 pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo Giometti <giometti@linux.it>
0.256000 clocksource: Switched to clocksource constant
0.264000 PS-cache: Loaded
0.444000 NET: Registered PF_INET protocol family
0.444000 IP ident hash table entries: 2048 (order: 2, 16384 bytes, linear)
0.456000 tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 4096 bytes, linear)
0.456000 TCP established hash table entries: 1024 (order: 0, 4096 bytes, linear)
0.456000 TCP bind hash table entries: 1024 (order: 0, 4096 bytes, linear)
0.460000 TCP: hash tables configured (established 1024 bind 1024)
0.460000 UDP hash table entries: 256 (order: 0, 4096 bytes, linear)
0.464000 UDP-Lite hash table entries: 256 (order: 0, 4096 bytes, linear)
0.464000 NET: Registered PF_UNIX/PF_LOCAL protocol family
0.504000 workingset: timestamp_bits=14 max_order=15 bucket_order=1
0.732000 IPMI message handler: version 39.2
0.732000 IPMI device interface
0.732000 ipmi_si: IPMI System Interface driver
0.760000 ipmi_si: unable to find any System Interface(s)
0.900000 Serial: 8250/16550 driver, 16 ports, IRQ sharing enabled
1.012000 i801e0.serial: ttyS0 at MMIO 0x1fe01e0 (irq = 18, base_baud = 2062500) is a 16550A
1.016000 random: Fast init done
1.216000 printk: console [ttyS0] enabled
1.264000 libphy: Fixed MDIO Bus: probed
1.288000 mousedev: ps/2 mouse device common for all mice
1.300000 IR MCE keyboard/mouse protocol handler initialized
1.308000 rfd: raw HID events driver (C) Jiri Kosina
1.348000 NET: Registered PF_INET6 protocol family
1.376000 Segment Routing with IPv6
1.384000 sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
6.240000 Freeing unused kernel image (initmem) memory: 4328K
6.240000 This architecture does not have kernel memory protection.
6.244000 Run /init as init process
can't run '/etc/init.d/rcS': No such file or directory

Please press Enter to activate this console.
/ # ls
a.sh      dev       init      linuxrc   results   usr
bin       emb       lib       media     sbin      var
chroot.sh etc       libp32    mnt       sys
chroot1.sh gen.sh    libp64    proc      tmp
/ #
  
```

图 6 Linux 启动示意图

7 参考设计说明

(1) 处理器优化设计参考了 2023 年西北工业大学团队赛参赛作品

(https://gitee.com/orangebird/serval_cat);

(2) 分支预测、AXI 转接桥和乘、除法器模块参考龙芯开源项目 openLA500 的设计;

(3) 流水级切分参考西北工业大学计算机学院王党辉教授的公开培训课程([2024 年“龙芯杯”培训 \(第二场\) CPU 性能优化技术](#));

(4) TLB 与 cache 相关接口参考《CPU 设计实战》^[4]中接口定义, cache 内部状态定义参考了龙芯开源项目 openLA500 的设计, 但是状态转换与其他信号为自主设计;

(5) 顺序双发射处理器部分结构设计参考 2024 年西北工业大学一队 gitee 开源仓库([NSCSCC-2024-team: 2024 龙芯杯西工大一队日常仓库](#))。

参考文献

[1]龙芯中科. 龙芯架构 32 位精简版参考手册 V1.03[EB/OL]. [2025-08-18]

[2]龙芯教育. chiplab[EB/OL]. [2025-08-18]. <https://gitee.com/loongson-edu/chiplab>

[3]姚永斌. 超标量处理器设计[M]. 北京:清华大学出版社,2014.

[4]汪文祥,邢金章.CPU 设计实战[M]. 北京:机械工业出版社,2021.

[5]龙芯教育.la32r-Linux[EB/OL].[2025-08-18].<https://gitee.com/loongson-edu/la32r-Linux>